

paper-agent 運用説明書

セットアップ・日常運用・メンテナンスガイド

Version: 1.0 | Date: 2026-06-02 | 前提: Python 3.10+, Claude Code (Pro または Max)、git

1. 初回セットアップ

Step 1 — リポジトリをクローン

```
git clone https://github.com/dan23514/paper-agent.git
cd paper-agent
```

Step 2 — Python 依存関係をインストール

```
pip install -r requirements.txt
```

Step 3 — 設定ファイルを作成

```
cp scripts/config.example.py scripts/config.py
```

`scripts/config.py` を開き、`MAILTO_EMAIL` を自分のメールアドレスに変更する（OpenAlex / CrossRef polite pool 用）。

Step 4 — SJR データを配置

1. <https://www.scimagojr.com/journalrank.php> にアクセス
2. ページ右下の「Download data」をクリックして CSV をダウンロード
3. ダウンロードしたファイルを `data/sjr_2025.csv` として保存（区切り文字はセミicolon）

Step 5 —（オプション）Exa API キーを設定

補完検索が必要な場合のみ。 `.claude/mcp.json` の `<EXA_API_KEY>` を実際のキーに置換する。

注意: API キーが入った状態で `git commit` しないこと。

1.1 動作確認

```
python scripts/sjr_filter.py "Journal of Oceanography"
# → Q2 と表示されれば正常
```

2. 新規プロジェクトの作成

2.1 入力ファイルを作成する

```
mkdir -p projects/my-paper/input
cp projects/template/input/input.md projects/my-paper/input/input.md
```

`projects/my-paper/input/input.md` をテキストエディタで開き、以下のセクションを記入する。

セクション	内容	推奨文字数
# Theme	研究タイトル候補	1～2行
# Background	研究背景・課題意識	200～800字
# Methods	使用した手法・実験条件	箇条書き可
# Data	データの概要（種類・規模・形式）	箇条書き可
# Results	主要な結果	200字～
# Interpretation	結果の解釈・考察の方向性	200字～
# Config	投稿先ジャーナル・引用スタイル・文字数	—

2.2 Config セクションの例

```
# Config
citation_target_journal: "Journal of Oceanography"
boost_journals:
  - name: "Journal of Oceanography"
    issn: "0916-8370"
length:
  abstract: 250
  body: 8000
citation_style: apa
```

3. チェックポイント（CP）別の操作手順

CP0: 入力受領

```
/load my-paper
```

- 入力ファイルを解析し `cp0_input.yaml` を生成する
- `config.yaml` と `progress.yaml`（進捗管理）を初期化する
- 欠損フィールドがあれば警告が表示される → 入力ファイルを補完して再実行

CP1: 文献検索 + Q1/Q2 フィルタ

```
/search my-paper
```

- OpenAlex API で検索（通常 30秒～2分）
- SJR データで Q1/Q2 フィルタを適用する
- CrossRef で全候補の BibTeX を取得する
- ユーザー確認: `cp1_references.md` を開いて不要な論文を削除・追加を確認する

SJR CSV が見つからない場合は「Step 4」を参照してダウンロード・配置する。

CP2: アウトライン生成

```
/outline my-paper
```

- IMRaD 構成のアウトラインを生成する
- ユーザー確認: `cp2_outline.md` を編集してセクション構成・引用割り当てを調整する

CP3: ドラフト執筆

```
/draft my-paper
```

- 本文・Abstract・Keywords をセクション別に執筆する（通常 5～15分）
- 引用 ID 検証（`verify_refs.py`）が自動実行される。無効 ID が検出された場合は自動再生成
- 途中で中断した場合は同コマンドを再実行すると `progress.yaml` から再開する

CP4: マルチ AI レビュー

```
/review-packet my-paper
```

- Claude 自動レビューが実行され `results/claude.md` が生成される
- ユーザー作業（詳細は `manuals/review_workflow.md` を参照）：
 1. ChatGPT に `prompts/chatgpt.md` + ドラフト全文を投稿し結果を `results/chatgpt.md` に保存
 2. Gemini に `prompts/gemini.md` + ドラフト全文を投稿し結果を `results/gemini.md` に保存

```
/integrate-review my-paper
```

- 3 AI のレビュー結果を統合し diff 形式で修正案を提示する
- 各修正について承認 / 却下を選択する

CP5: Factcheck

```
/factcheck-packet my-paper
```

- 本文中の引用主張を抽出し `claims.md` を生成する
- OA-PDF の自動ダウンロードを試みる（PDF マジックバイト検証付き）
- ユーザー作業（詳細は `manuals/notebooklm_workflow.md` を参照）：
 1. `missing_pdfs.md` の論文を機関アクセスで入手し `pdfs/` に配置する
 2. NotebookLM に PDF 全件をアップロードする（上限 50件）
 3. `notebooklm_prompts.md` のプロンプトを投げて `results.md` に Verdict を記入する

```
/integrate-factcheck my-paper
```

- NOT_SUPPORTED / PARTIAL の引用を本文に反映する

CP7: Word 出力

```
/export my-paper
```

- 引用を APA/IEEE 等の実形式に変換し参考文献リストを付加する
- `python-docx` で `output/*.docx` を生成する
- 出力先: `projects/my-paper/output/`

4. エラー回復手順

4.1 途中中断からの再開

同じコマンドを再実行するだけで `progress.yaml` から再開する。

4.2 特定の CP に巻き戻す

`projects/my-paper/checkpoints/progress.yaml` を直接編集する。

```
current_cp: 2          # ← ここを変更（例: 2 = CP2 から再開）
cp3_sections_done: []
cp3_sections_pending: [introduction, methods, results, discussion, conclusion, abstract, keywords]
```

4.3 よくあるエラーと対応

エラー	原因	対応
SJR CSV not found	data/sjr_YYYY.csv が存在しない	Scimago からダウンロードして配置
ModuleNotFoundError: yaml	pip install 未実施	<code>pip install -r requirements.txt</code>
OpenAlex returns 0 results	クエリが長すぎる / 英語でない	短い英語キーワードに変更
PDF 取得後に中身が HTML	OA リンクがランディングページを返す	手動で機関アクセス PDF を <code>pdfs/</code> に配置 (pdf_fetcher は PDF マジックバイト検証済み)
INVALID IDs in citation guard	Writer が候補リスト外の ID を生成した	コマンドが自動再生成する（最大2回）。2回失敗後は <code>error_log</code> を確認

5. 定期メンテナンス

5.1 SJR データの年次更新

毎年 1～2月に最新年の CSV が公開される。

1. <https://www.scimagojr.com/journalrank.php> から新しい年の CSV をダウンロード
2. `data/sjr_YYYY.csv` (例: `sjr_2026.csv`) として配置
3. `scripts/config.py` の `SJR_YEAR = 2026` に更新

5.2 誌名略語辞書の追加

SJR マッチングで認識されない雑誌がある場合:

```
python scripts/add_abbr.py "Nat. Commun." "Nature Communications"
```

5.3 journal_abbr.json のコミット

略語を追加したら GitHub にコミットしておく、他の環境でも引き継がれる。

```
git add data/journal_abbr.json
git commit -m "chore: add journal abbreviation entries"
git push origin main
```

6. コミット・バージョン管理ポリシー

- コミットして良いもの: `.claude/`、`scripts/` (`config.py` を除く)、`manuals/`、`docs/`、`data/journal_abbr.json`、`CLAUDE.md`、`README.md`、`architecture.md`、`requirements.txt`
- コミットしてはいけないもの: `config.py` (メール)、SJR CSV、PDF、`projects/` 内の研究データ
- コミット前に `git status` で想定外のファイルが含まれていないか確認すること

7. 複数の論文プロジェクトの管理

プロジェクトは `projects/` 以下に名前ごとに独立して作成できる。

```
projects/
├─ jstf-ver1/    # 日本海通過流 CMIP6 解析
├─ co2-uptake/   # 別の論文プロジェクト
└─ template/    # 入力テンプレート (コピー元)
```

各プロジェクトは独自の `config.yaml` ・ `progress.yaml` ・ `checkpoints/` を持つため、並行して管理できる。
コマンドの引数にプロジェクト名を指定するだけで切り替わる。

8. 学会テンプレートの利用

1. 学会の Word テンプレート (.docx) を `templates/<学会名>.docx` に配置する
2. `projects/my-paper/config.yaml` の `export.template` にパスを指定する:

```
export:  
  template: "templates/oceanography_template.docx"
```

テンプレートのスタイル定義（禁則処理・フォント・段落設定）を継承して出力する。